Project-3

1. **Clone the git repo.**

```
buntu@ip-172-31-37-161:~$ ls
rend  awscliv2.zip  devops-build
ws    cluster.yaml  eks-cluster.yaml
buntu@ip-172-31-37-161:~$
```

2. **Creating dockerfile.**

ubuntu@ip-172-31-37-161: ~/devops-build

```
# Use a lightweight Nginx image
FROM nginx:stable-alpine

# Copy the pre-existing build folder content to the Nginx html directory
COPY build/ /usr/share/nginx/html/

# Expose port 80
EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
~
~
~
~
~
~
~
~
~
~
~
```

3. **creating build.sh**

ubuntu@ip-172-31-37-161: ~/devops-build

```
#!/bin/bash

IMAGE_NAME=my-react-app
DOCKERHUB_USER=yourusername
TAG=dev

docker pull $DOCKERHUB_USER/$IMAGE_NAME:$TAG

docker stop myapp || true
docker rm myapp || true

docker run -d -p 80:80 --name myapp $DOCKERHUB_USER/$IMAGE_NAME:$TAG
~
~
~
~
~
~
~
~
```

4. **creating deploy.sh**

ubuntu@ip-172-31-37-161: ~/devops-build

```
#!/bin/bash

# These variables make the script flexible
IMAGE_NAME=$1
DOCKERHUB_USER=$2
TAG=$3
DOCKER_PASS=$4

# 1. Login to DockerHub (Required for the Private Prod repo)
echo "$DOCKER_PASS" | docker login -u "$DOCKERHUB_USER" --password-stdin

# 2. Pull the latest image
docker pull "$DOCKERHUB_USER/$IMAGE_NAME:$TAG"

# 3. Clean up ANY container using port 80 to avoid the "Bind failed" error
# We remove 'myapp' (from your script) and 'my-react-app-prod' (from Jenkins)
docker rm -f myapp my-react-app-prod || true

# 4. Run the new container
docker run -d -p 80:80 --name my-react-app-prod "$DOCKERHUB_USER/$IMAGE_NAME:$TAG"
~
~
~
~
~
```

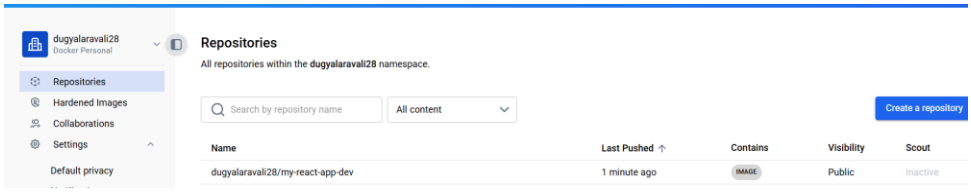
5. create dockerignore.

```
ubuntu@ip-172-31-37-161: ~/devops-build
node_modules
.git
.gitignore
Dockerfile
README.md
*.log
.env
```

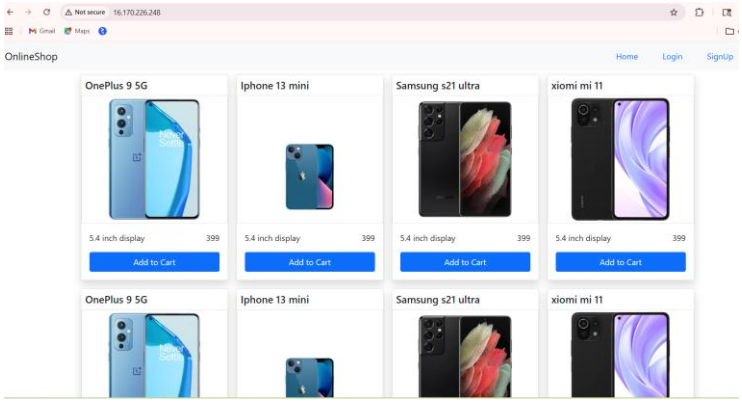
6. build and push the docker image

```
ubuntu@ip-172-31-37-161:~/devops-build$ docker build -t my-react-app .
docker run -d -p 80:80 my-react-app
DEPRECATED: The legacy builder is deprecated and will be removed in a future rel
ease.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.625MB
Step 1/4 : FROM nginx:stable-alpine
 ---> 0272e4604ed9
Step 2/4 : COPY build/ /usr/share/nginx/html/
 ---> Using cache
 ---> b82df3ba3bf7
Step 3/4 : EXPOSE 80
 ---> Using cache
 ---> 1a66c1989e2a
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
 ---> Using cache
 ---> 8769a9044451
Successfully built 8769a9044451
Successfully tagged my-react-app:latest
f0788a28603851f41b801dd257c537de378c55a5f8b29de8e926ca264508d633
ubuntu@ip-172-31-37-161:~/devops-build$
```

7. dockerhub repo

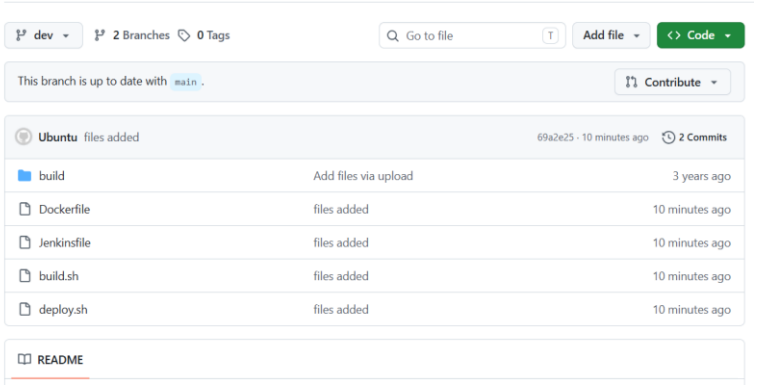


8. test locally

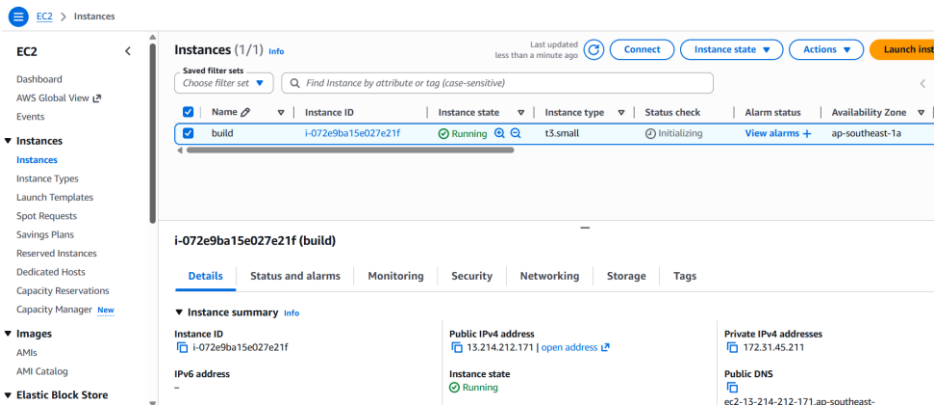


9. push the code to dev branch.

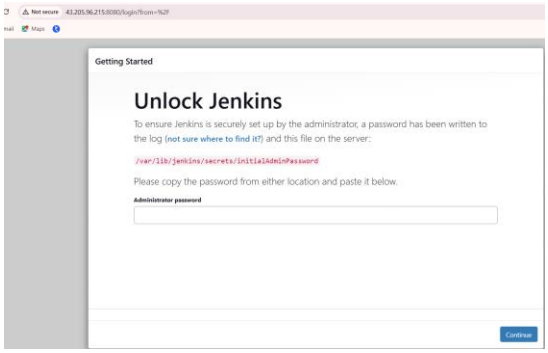
```
branch 'main' set up to track 'origin/main'.
ubuntu@ip-172-31-37-161:~/devops-build$ git checkout -b dev
git push -u origin dev
Switched to a new branch 'dev'
Username for 'https://github.com': Dugyalaravali
Password for 'https://Dugyalaravali@github.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'dev' on GitHub by visiting:
remote:      https://github.com/Dugyalaravali/devops-build/pull/new/dev
remote:
To https://github.com/Dugyalaravali/devops-build.git
 * [new branch]      dev -> dev
branch 'dev' set up to track 'origin/dev'.
ubuntu@ip-172-31-37-161:~/devops-build$
```



10. Depoly server lunched.



11. Ec2 lunched and jenkin installed



12. Login to jenkin server

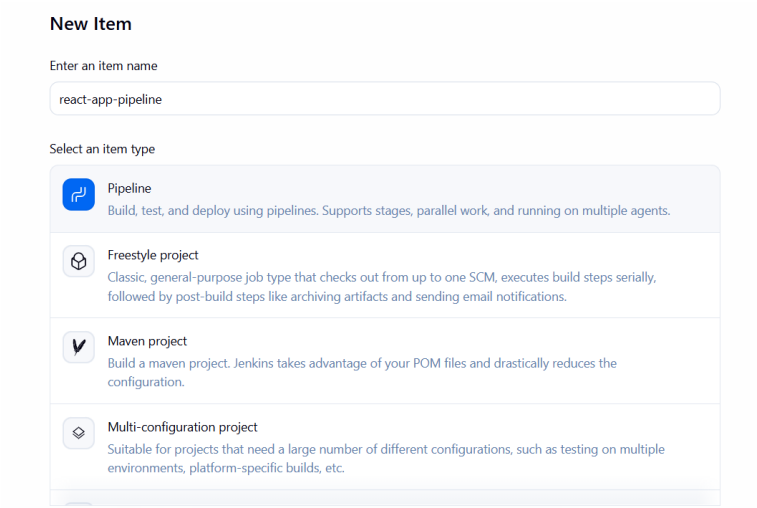
```
ubuntu@ip-10-0-1-139:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
71aba6a5a3f749728d73516de4174bcf
ubuntu@ip-10-0-1-139:~$
```

13. Creating jenkinsfile

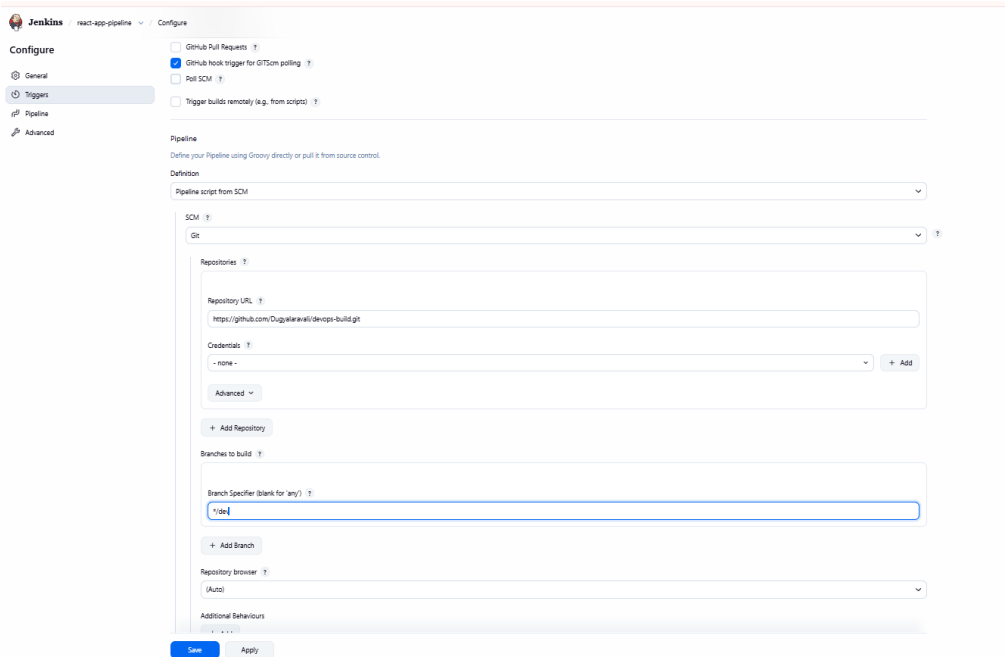
```
pipeline {
    agent any

    environment {
        // Docker Hub details
        DOCKERHUB = "dugyalaravali28"
        DEV_IMAGE = "my-react-app-dev"
        PROD_IMAGE = "my-react-app-prod"

        // Deployment Server details
        DEPLOY_SERVER = "13.214.212.171"
    }

    stages {
        stage('Build & Push Dev') {
            steps {
                // Build the image using the Dev repository name
                sh 'docker build -t $DOCKERHUB/$DEV_IMAGE:dev .'

                withCredentials([usernamePassword(
                    credentialsId: 'docker-creds',
                    usernameVariable: 'USER',
                    passwordVariable: 'PASS'
                )]) {
                    // Login to Docker Hub
                    sh 'echo $PASS | docker login -u $USER --password-stdin'
                    // Push to the public dev repository
                    sh 'docker push $DOCKERHUB/$DEV_IMAGE:dev'
                }
            }
        }

        stage('Promote to Prod Repo') {
            steps {
                // Retag the verified dev image to the prod repository name
                sh """
                    docker tag $DOCKERHUB/$DEV_IMAGE:dev $DOCKERHUB/$PROD_IMAGE:latest
                    docker push $DOCKERHUB/$PROD_IMAGE:latest
                """
            }
        }

        stage('Deploy to Prod Server') {
            steps {
                // Use SSH agent for secure key handling
                sshagent(['deploy-server-key']) {
                    withCredentials([usernamePassword(
                        credentialsId: 'docker-creds',
                        usernameVariable: 'USER',
                        passwordVariable: 'PASS'
                    )]) {
                        sh """
                            ssh -o StrictHostKeyChecking=no ubuntu@${DEPLOY_SERVER} << EOF
                                # 1. Log in on the remote server to pull from the PRIVATE repo
                                echo ${PASS} | docker login -u ${USER} --password-stdin

                                # 2. Pull the latest Production image
```

```
        stage('Promote to Prod Repo') {
            steps {
                // Retag the verified dev image to the prod repository name
                sh """
                    docker tag $DOCKERHUB/$DEV_IMAGE:dev $DOCKERHUB/$PROD_IMAGE:latest
                    docker push $DOCKERHUB/$PROD_IMAGE:latest
                """
            }
        }

        stage('Deploy to Prod Server') {
            steps {
                // Use SSH agent for secure key handling
                sshagent(['deploy-server-key']) {
                    withCredentials([usernamePassword(
                        credentialsId: 'docker-creds',
                        usernameVariable: 'USER',
                        passwordVariable: 'PASS'
                    )]) {
                        sh """
                            ssh -o StrictHostKeyChecking=no ubuntu@${DEPLOY_SERVER} << EOF
                                # 1. Log in on the remote server to pull from the PRIVATE repo
                                echo ${PASS} | docker login -u ${USER} --password-stdin

                                # 2. Pull the latest Production image
                                docker pull ${DOCKERHUB}/${PROD_IMAGE}:latest

                                # 3. CLEANUP: Force remove any container using Port 80
                                # This prevents the "Port is already allocated" error
                                docker rm -f \\\$(docker ps -q --filter "publish=80") || true

                                # Also remove by specific names used previously
                                docker rm -f myapp my-react-app-prod || true

                                # 4. Run the new container on Port 80
                                docker run -d --name my-react-app-prod -p 80:80 ${DOCKERHUB}/${PROD_IMAGE}:latest
EOF
                        """
                    }
                }
            }
        }
    }

    post {
        always {
            // Clean up the workspace after the build
            cleanWs()
        }
    }
```

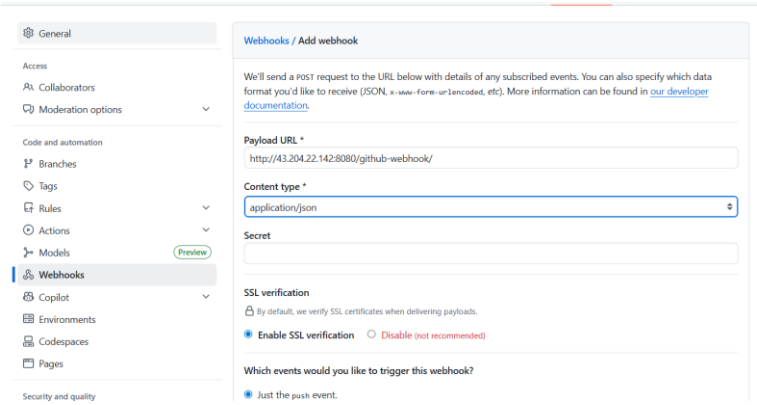
14. Setup clcd pipeline.



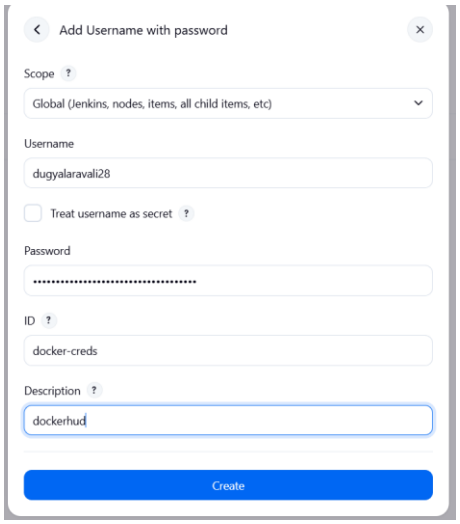
Adding github repo details



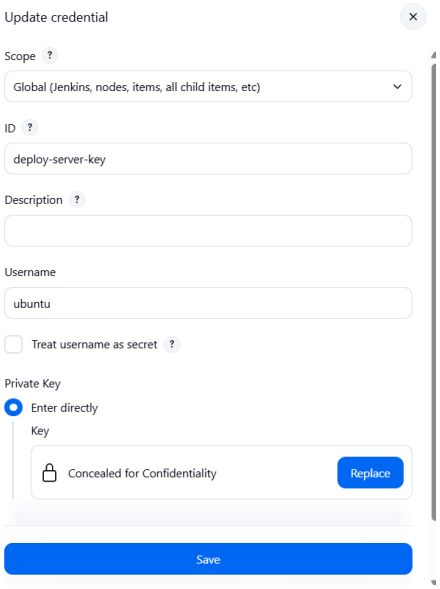
Creating webhook.



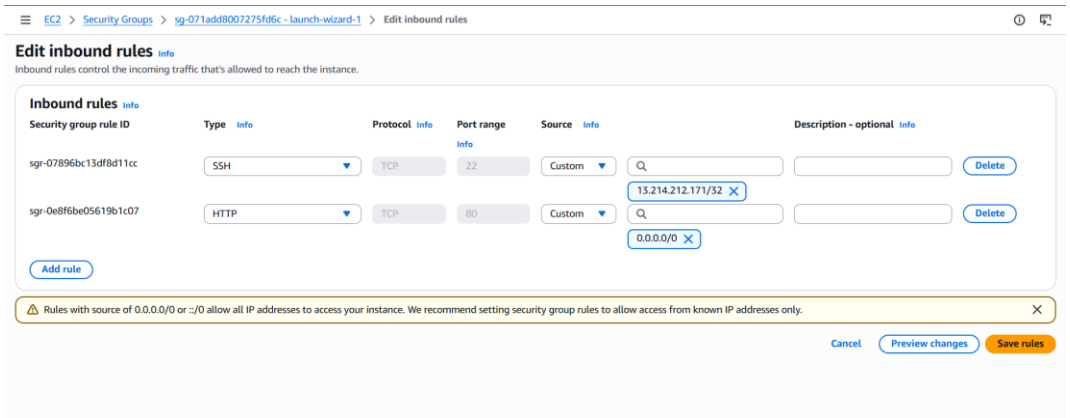
Adding dockerhub credentials.



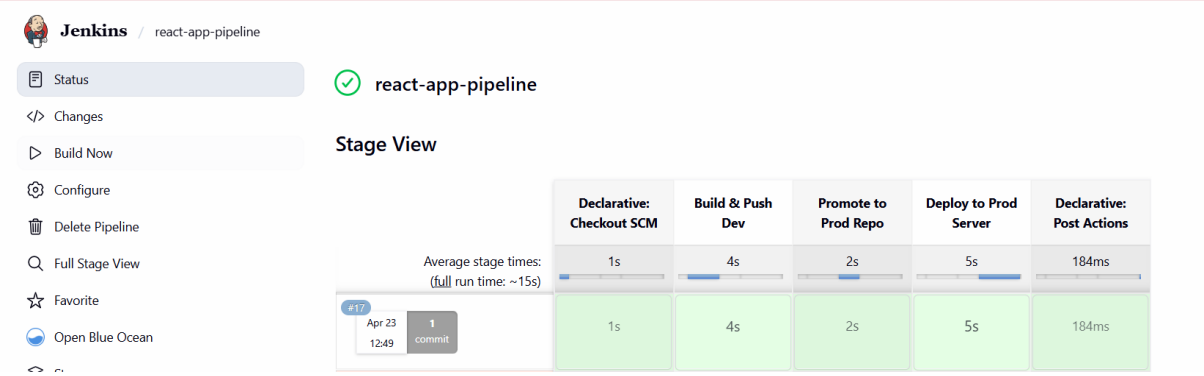
Added deploy server ssh key



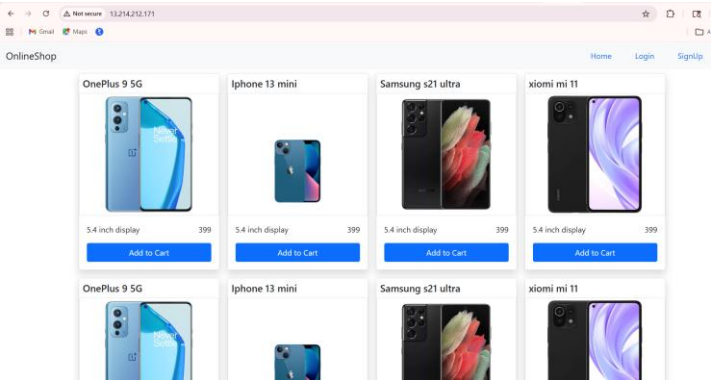
15. **Added Jenkins server ip in deploy server inbound rules.**



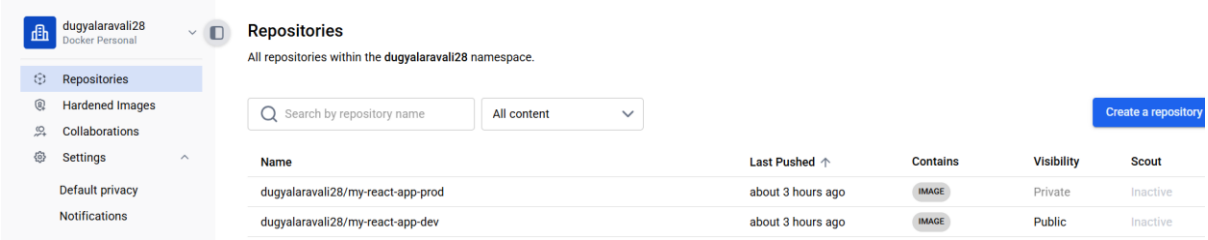
16. Succefully deployed



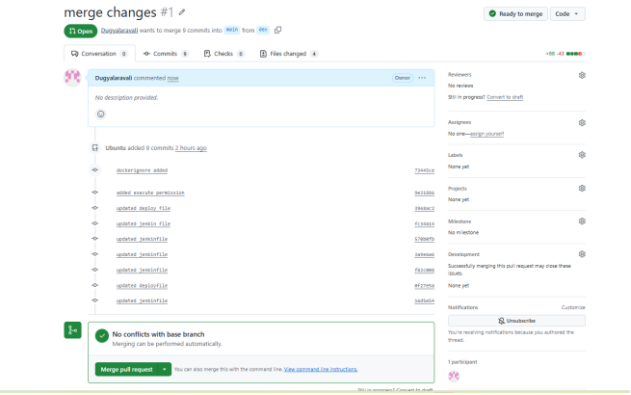
17. Succefully reached deploy server

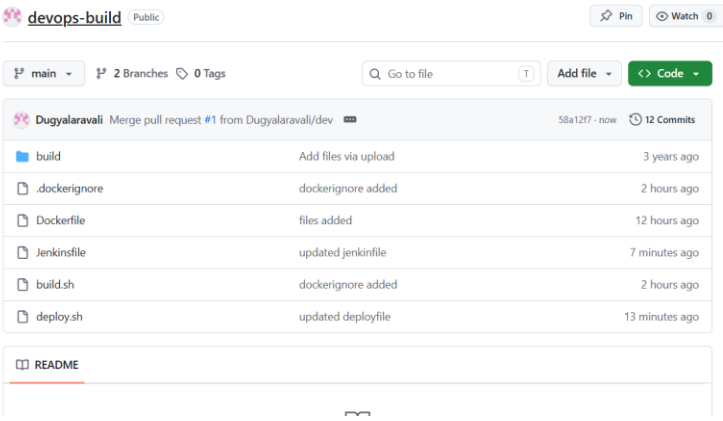


18. Dockerhub public and private repos.

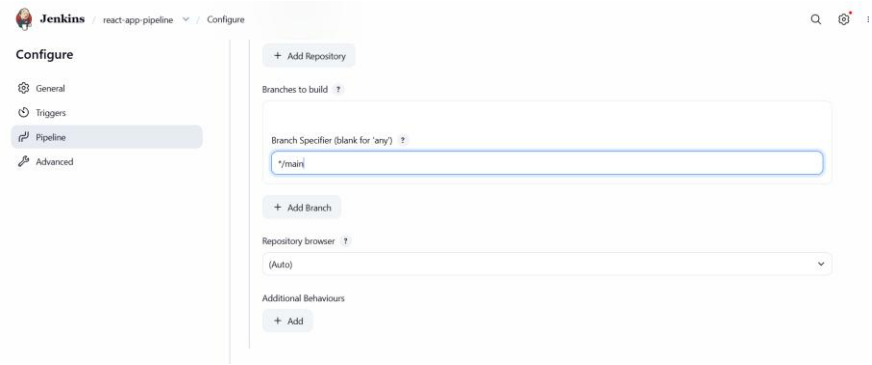


19. Create merge to main branch.

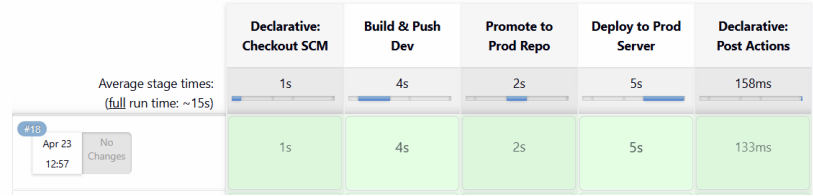




19. Trigged main branch pipeline



20. Completed successfully



21.setup monitoring tools

```
ubuntu@ip-172-31-45-211:~$ cd monitoring
ubuntu@ip-172-31-45-211:~/monitoring$ ls
docker-compose.yml  prometheus.yml
ubuntu@ip-172-31-45-211:~/monitoring$ |
```

Creating Prometheus yaml file

```
ubuntu@ip-172-31-45-211: ~/monitoring
global:
  scrape_interval: 15s

scrape_configs:
  # Check the health of your React App
  - job_name: 'react-app-health'
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets:
        - http://13.214.212.171 # Your App IP
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - target_label: __address__
        replacement: blackbox-exporter:9115 # Points to the exporter container
~
~
~
~
~
~
~
~
~
~
```

Creating docker compose

ubuntu@ip-172-31-45-211: ~/monitoring

```
version: '3.8'

services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    restart: always

  blackbox-exporter:
    image: prom/blackbox-exporter:latest
    container_name: blackbox-exporter
    ports:
      - "9115:9115"
    restart: always

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    restart: always
~
~
~
~
~
~
~
~
~
```

Run docker compose up

```
ubuntu@ip-172-31-45-211:~/monitoring$ docker compose up -d
WARN[0000] /home/ubuntu/monitoring/docker-compose.yml: the attribute `version` i
s obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 3/3
 ✔ Container grafana           Running                                    0.0s
 ✔ Container prometheus        Running                                    0.0s
 ✔ Container blackbox-exporter Started                                    0.1s
ubuntu@ip-172-31-45-211:~/monitoring$ Read from remote host 13.214.212.171: Connection reset by peer
Connection to 13.214.212.171 closed.
client_loop: send disconnect: Connection reset by peer
```

21. Add monitoring ports in deploy server

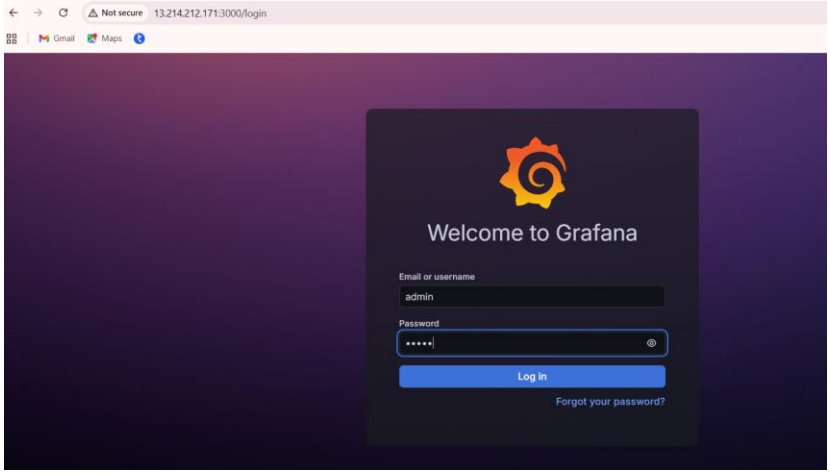
EC2 > Security Groups > sg-071add8007275fd6c - launch-wizard-1 > Edit inbound rules

Inbound rules Info

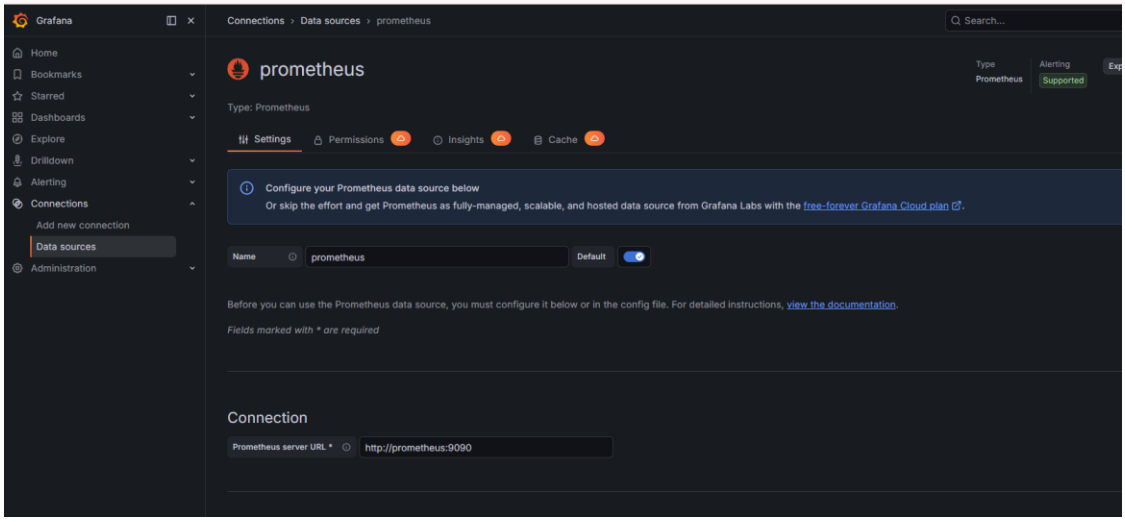
Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info		Description - optional Info	
sgr-07896bc13df8d11cc	SSH	TCP	22	Custom	0.0.0.0/0		Delete
sgr-0e8f6be05619b1c07	HTTP	TCP	80	Custom	0.0.0.0/0		Delete
-	Custom TCP	TCP	9090	Anywh...	0.0.0.0/0		Delete
-	Custom TCP	TCP	3000	Anywh...	0.0.0.0/0		Delete

Add rule

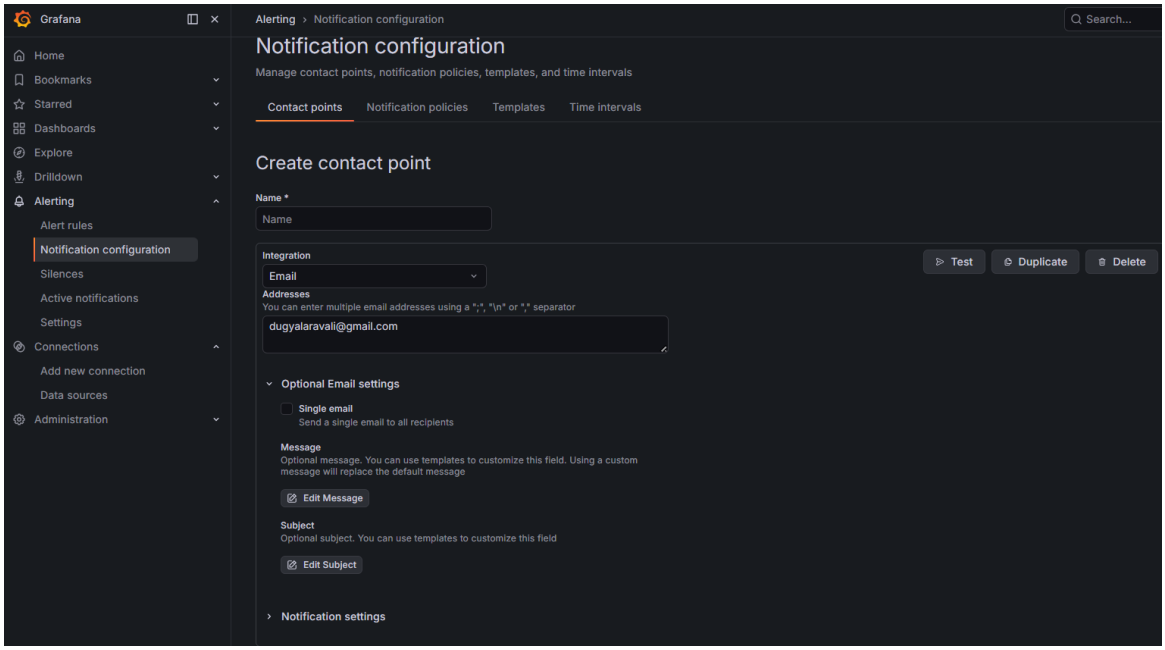
22. Login to Grafana.



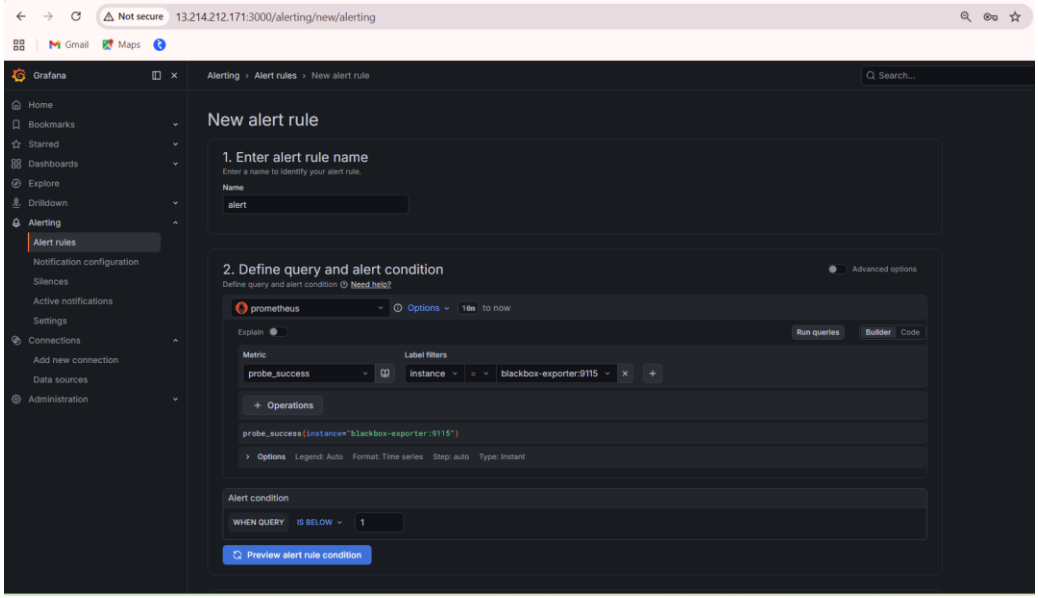
Integrate Prometheus in Grafana

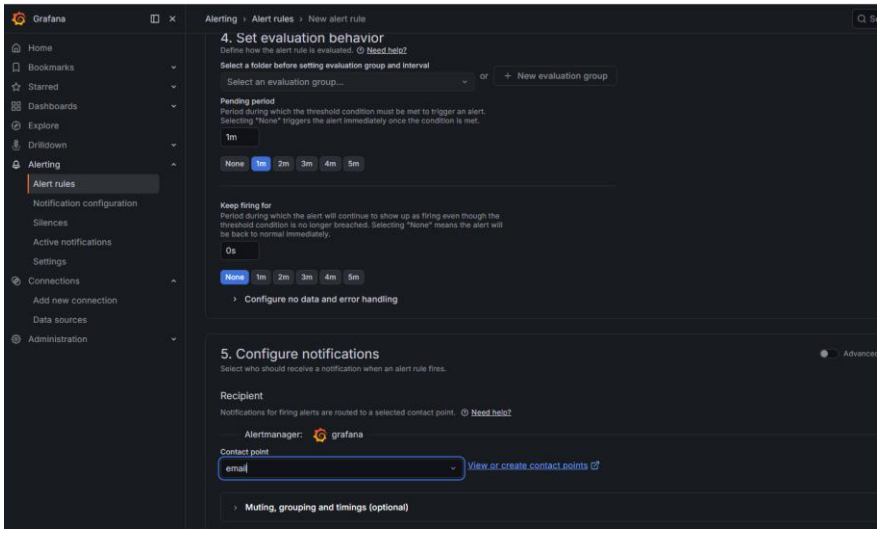


Create contanct points

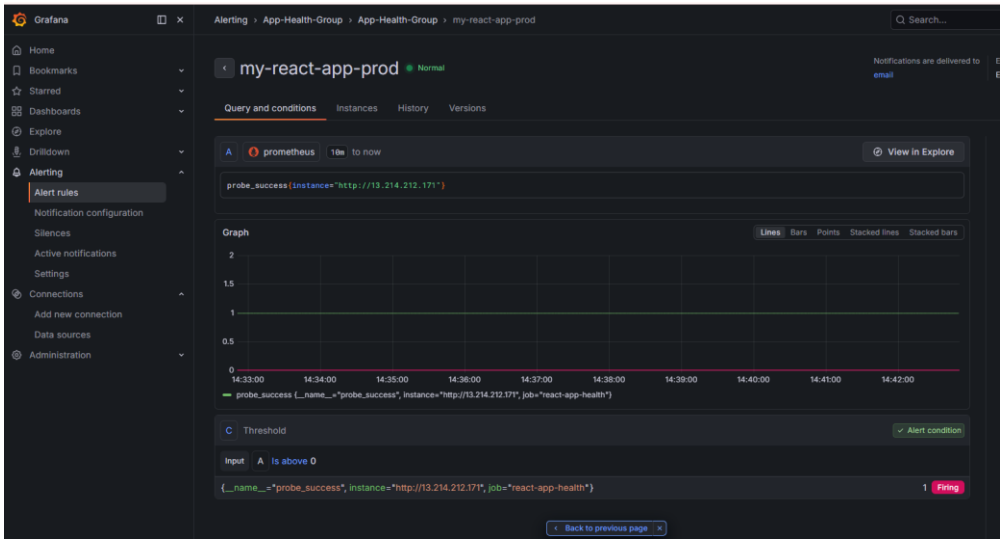
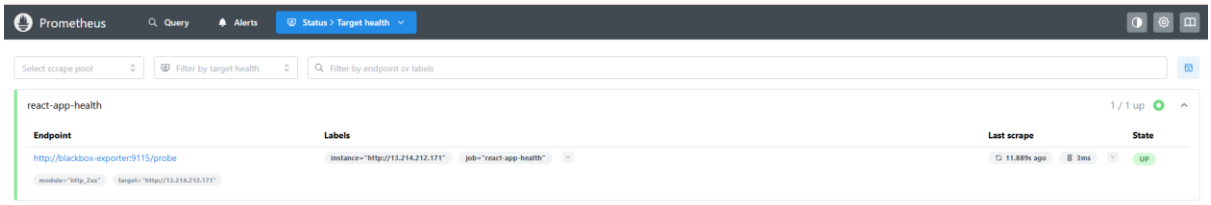


Creating alerts.





Metrics



23. **Prometheus status**